Chatbot do Trabalho Garantido

Projeto que visa desenvolver um chatbot para responder dúvidas sobre o programa Trabalho Garantido, oferecido pela FMU aos alunos ingressantes no primeiro semestre de seus cursos.

O Programa



O Trabalho Garantido é um programa da FMU voltado à preparação e inserção dos estudantes no mercado de trabalho. Por meio de ações de qualificação, orientação profissional e encaminhamento para processos seletivos, o programa busca ampliar a empregabilidade dos participantes e facilitar sua entrada profissional na área de formação ou em áreas correlatas.

Visão geral do projeto

O projeto consiste em um assistente virtual baseado em RAG, desenvolvido para responder dúvidas sobre o edital do programa Trabalho Garantido. O sistema localiza no documento os trechos mais relevantes para a pergunta e os envia como contexto para o LLM, que gera a resposta final em linguagem natural.

Arquitetura e tecnologias do projeto

- **RAG** (Retrieval-Augmented Generation) – arquitetura que combina recuperação de informações do edital com geração de respostas por um LLM.
- **Embeddings** – transformam os trechos do documento e a pergunta do usuário em representações vetoriais, permitindo comparar seus significados.
- **Chroma** – banco vetorial responsável por armazenar os trechos vetorizados e recuperar os mais relevantes para cada pergunta.
- **LLM** – GPT-OSS-120B – modelo de linguagem que recebe a pergunta e os trechos recuperados para elaborar a resposta em linguagem natural.
- **Groq** – infraestrutura/API utilizada para executar o LLM.
- **Streamlit** – framework utilizado para desenvolver a interface web do chatbot.
- **LangChain** – utilizado para integrar e organizar as diferentes etapas do fluxo RAG.

Arquitetura do Chatbot Trabalho Garantido



Usuário

Faz a pergunta na interface.

1 Interface web

Streamlit

Recebe a pergunta do usuário.

2 Embeddings

A pergunta é transformada em uma representação vetorial (embedding).

3 Busca no banco vetorial

Chroma

Compara o vetor da pergunta com os trechos do edital armazenados e recupera os mais relevantes.

4 Geração da resposta com LLM

groq

GPT-OSS-120B

Os trechos recuperados e a pergunta são enviados ao LLM, que gera a resposta em linguagem natural.

5 Resposta

A resposta é exibida n a interface, com informações baseadas no edital (podendo incluir as fontes).

PDF

Edital Trabalho Garantido

Divisão em trechos (chunks)

Trechos vetorizados e armazenados no Chroma

Principais tecnologias

Streamlit
Interface web

LangChain
Integra as etapas do fluxo RAG

Chroma
Banco vetorial

Embeddings
Transformação de textos em vetores

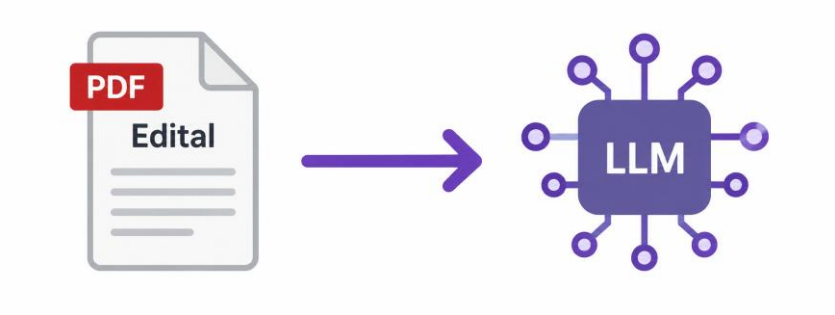
GPT-OSS-120B
Modelo de linguagem (LLM)

RAG
Arquitetura que combina recuperação de informações e geração de respostas

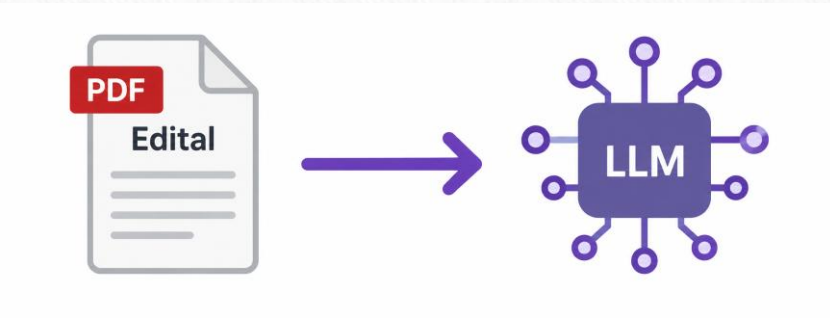
Arquitetura	Tecnologias empregadas
RAG – arquitetura principal do sistema	LangChain – framework utilizado para integrar os componentes do RAG
LLM – modelo de linguagem usado para geração da resposta em linguagem natural	GPT-OSS-120B
Embeddings – representação vetorial dos textos	HuggingFace Embeddings – implementação usada para gerar os embeddings
Busca semântica / recuperação – localização dos trechos relevantes	Chroma – realiza buscas por similaridade vetorial
Chunking – divisão do edital em trechos	RecursiveCharacterTextSplitter – componente do LangChain
Banco vetorial – armazenamento e consulta dos vetores	Chroma – banco de dados vetorial
Interface conversacional – interação usuário ↔ sistema	Streamlit – construção da aplicação web

RAG

RAG (Retrieval-Augmented Generation) – Geração Aumentada por Recuperação – é uma arquitetura que combina a capacidade de geração de texto de um LLM com a recuperação de informações provenientes de uma fonte externa. No projeto Trabalho Garantido, essa fonte externa é o **edital do programa**.



RAG



- **Retrieval** → Recuperação (ou busca) de informações
- **Augmented** → Aumentada / enriquecida
- **Generation** → Geração

Ou seja, a ideia do nome é: a geração da resposta pelo LLM é enriquecida pelas informações que o sistema recuperou de uma fonte externa — no nosso caso, o edital.

Fluxo do chatbot

Posso continuar no programa se ficar com dependência em uma disciplina?



Usuário

PDF

Edital Trabalho Garantido

PROGRAMA TRABALHO GARANTIDO FMU LEVEL UP

Busca dos trechos relevantes no edital

Recebe a pergunta e os trechos recuperados do edital e gera a resposta em linguagem natural.

LLM

GPT-OSS-120B (via Groq)

Resposta

Não. De acordo com o edital, para permanecer no programa, o aluno não gode ter disciplinas em dependência.

Fonte
Edital Trabalho Garantido
Página 4
(Seção 3.2 – Critérios de permanência)

1 Pergunta

O usuário faz uma pergunta sobre o programa na interface do chatbot.

2 Fonte de dados

O sistema utiliza o edital do Trabalho Garantido (PDF) como base de conhecimento.

3 Recuperação

O sistema divide o edital em trechos (**chunks**), transforma em vetores e busca os trechos mais relevantes para a pergunta.

4 Geração da resposta

O LLM recebe a pergunta e os trechos recuperados e gera a resposta em linguagem natural.

5 Resposta final

A resposta é exibida na interface, junto com as fontes (arquivo e página) dos trechos utilizados.

Por que utilizar RAG?

- Um LLM (chatgpt, gemini, claude, etc.) possui conhecimento adquirido durante seu treinamento, mas não conhece necessariamente documentos específicos ou informações privadas e atualizadas de uma organização. O RAG permite fornecer ao modelo informações externas relevantes antes da geração da resposta, tais como:
  - **Acesso a conhecimento específico** – permite utilizar documentos próprios, como o edital do Trabalho Garantido.
  - **Respostas contextualizadas** – o LLM responde considerando os trechos recuperados do documento.
  - **Redução de alucinações** – diminui a possibilidade de respostas inventadas ou sem base na fonte.
  - **Rastreabilidade** – permite apresentar as fontes e páginas utilizadas na elaboração da resposta.

Por que utilizar RAG?

- **Sem RAG:** pergunta → LLM → resposta baseada no conhecimento do modelo
- **Com RAG:** pergunta → **busca no edital** → LLM → resposta baseada no edital

Mercado de trabalho

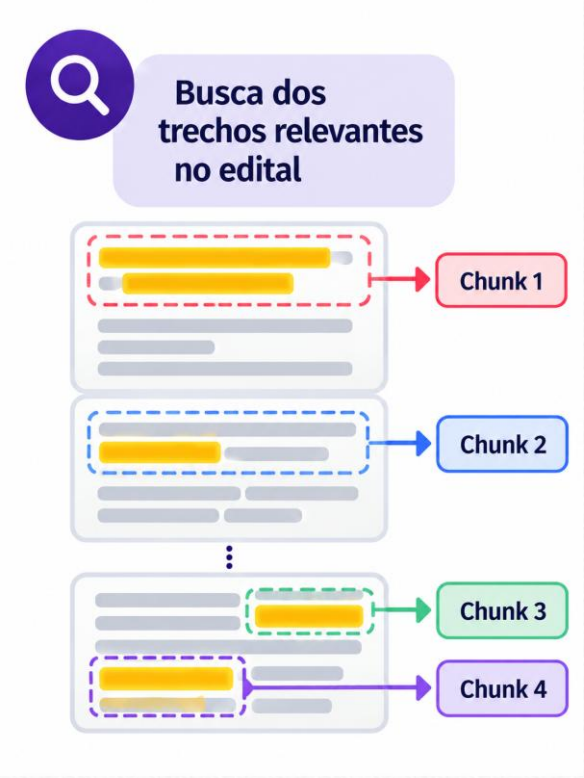
RAG é usado em assistentes corporativos, atendimento, busca inteligente, suporte técnico, análise de documentos, jurídico, educação e sistemas internos que precisam responder com base em conteúdo próprio da organização. Conhecer RAG aproxima o aluno de tarefas de engenharia de IA e desenvolvimento de aplicações com LLMs.

Chunking – divisão do documento

- Um documento extenso não é processado como um único bloco. Ele é dividido em trechos menores, chamados **chunks**, que serão utilizados posteriormente na busca pelas informações mais relevantes.

`PyPDFLoader` → carrega/extrai o texto do PDF

`RecursiveCharacterTextSplitter` → divide o texto em chunks



Chunking – divisão do documento

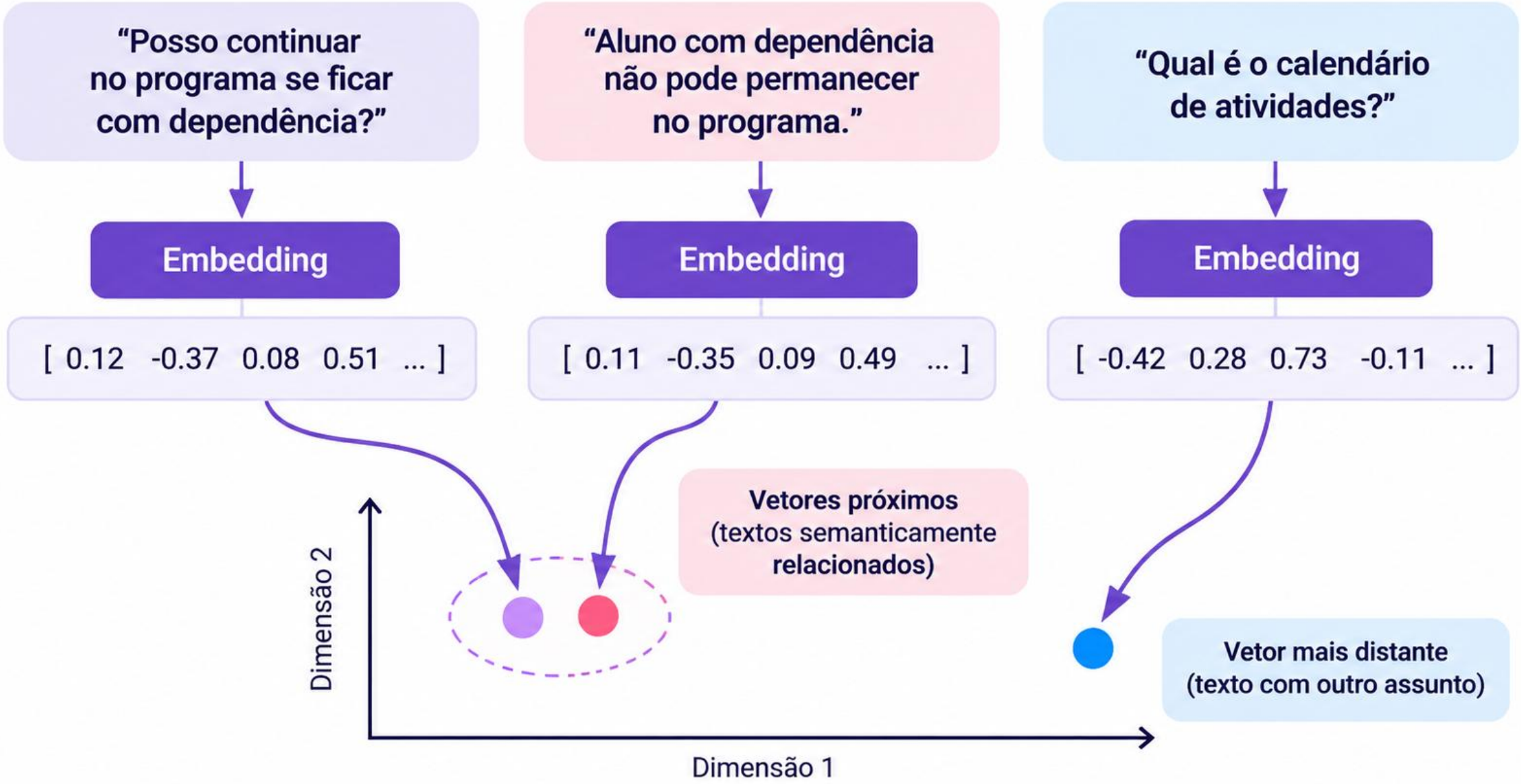
- **Ligação com o mercado de trabalho**
  - Definir boas estratégias de chunking é uma habilidade prática em projetos de RAG. O tamanho ideal depende do tipo de documento, do modelo de embeddings e da natureza das perguntas.

```
RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
```

O edital é dividido em blocos de até 1000 caracteres, com sobreposição de 200 caracteres.

Embeddings

- Embeddings são representações numéricas vetoriais de textos. Textos semanticamente semelhantes tendem a produzir vetores próximos no espaço vetorial.
- Eles permitem realizar busca por significado, e não apenas pela ocorrência exata das mesmas palavras. São uma ponte importante entre PLN e sistemas modernos de recuperação semântica.
- **Ligação com o mercado de trabalho**
  - Embeddings são usados em busca semântica, recomendação, clustering, deduplicação, classificação e RAG. Entender esse conceito é relevante para funções ligadas a PLN e engenharia de IA.


"Posso continuar no programa se ficar com dependência?"
"Aluno com dependência não pode permanecer no programa."
"Qual é o calendário de atividades?"
Embedding
Embedding
Embedding
[0.12 -0.37 0.08 0.51 ...]
[0.11 -0.35 0.09 0.49 ...]
[-0.42 0.28 0.73 -0.11 ...]
Vetores próximos (textos semanticamente relacionados)
Vetor mais distante (texto com outro assunto)
Dimensão 2
Dimensão 1

Chroma — banco vetorial

- Após a criação dos embeddings, os vetores dos chunks são armazenados em um **banco vetorial**.
- Bancos vetoriais são diferentes de bancos relacionais tradicionais: seu foco é encontrar vetores próximos, o que possibilita busca semântica eficiente.

Por exemplo:

Pergunta: “Posso ficar no programa se tiver DP?”
O edital pode não usar a expressão “DP”, mas conter:
“ausência de dependências ou adaptações acumuladas...”

```
banco_vetorial = Chroma.from_documents(blocos_texto, embeddings)
retriever = banco_vetorial.as_retriever(search_kwargs={"k": 5})
```

Ligação com o mercado de trabalho

Vector databases e mecanismos de busca vetorial aparecem em aplicações de IA generativa, recomendação e pesquisa semântica. Outros produtos do mercado seguem conceitos semelhantes.

LLM — Large Language Model

- LLM é um modelo de linguagem de grande escala treinado para processar e gerar linguagem natural. No projeto, ele recebe a pergunta e o contexto recuperado pelo RAG e produz a resposta final.
- O LLM oferece flexibilidade linguística: consegue sintetizar os trechos recuperados e produzir uma resposta natural, em vez de apenas retornar o trecho bruto.

temperature=0, representa o grau de variação na resposta, quanto mais baixa, mais precisão.

uso no projeto

```
llm = ChatGroq(
    model="openai/gpt-oss-120b",
    temperature=0
)
```

Ligação com o mercado de trabalho

LLMs estão sendo incorporados a atendimento, produtividade, programação, análise documental, educação e produtos digitais. Profissionais precisam saber integrar modelos, avaliar respostas e controlar contexto, custo e segurança.

Prompt Engineering

- É o processo de estruturar essas instruções para orientar o comportamento do modelo.
- Mesmo com um bom RAG, o modelo precisa saber como usar o contexto. Instruções claras podem exigir que ele utilize as fontes fornecidas e reconheça quando a informação não está disponível.

```
system_prompt = (
    "Você é um assistente virtual especializado no Edital do Programa Trabalho Garantido da FMU.\n"
    "Use estritamente os trechos do edital fornecidos abaixo para responder à pergunta do usuário.\n"
    "Se não souber a resposta ou se ela não estiver no texto, diga honestamente que não encontrou.\n\n"
    "Contexto do edital:\n{context}"
)
```

Hugging Face e Sentence Transformers

- **Hugging Face** é um ecossistema amplamente utilizado para modelos, datasets e ferramentas de Machine Learning e PLN. **Sentence Transformers** é uma família/biblioteca voltada à geração de embeddings de sentenças e textos.
- Permite utilizar modelos de embeddings prontos, evitando treinar um modelo do zero para representar semanticamente os textos.

Hugging Face e Sentence Transformers

- **Hugging Face** é um ecossistema amplamente utilizado para modelos, datasets e ferramentas de Machine Learning e PLN. **Sentence Transformers** é uma família/biblioteca voltada à geração de embeddings de sentenças e textos.
- Permite utilizar modelos de embeddings prontos, evitando treinar um modelo do zero para representar semanticamente os textos.

```
HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

Ligação com o mercado de trabalho

- O ecossistema Hugging Face é relevante em pesquisa e desenvolvimento aplicado de IA. Saber localizar, avaliar e integrar modelos pré-treinados é uma competência prática importante.

https://huggingface.co/

Groq e API

- APIs permitem que uma aplicação utilize serviços e modelos hospedados externamente. O desenvolvedor não precisa executar localmente toda a infraestrutura computacional necessária para um LLM grande.
- **Groq** é a plataforma de API utilizada pelo projeto para executar o LLM. É importante distinguir o provedor da API do modelo de linguagem: a Groq fornece a infraestrutura; o **GPT-OSS-120B** é o modelo que gera o texto.

Ligação com o mercado de trabalho

Integração com APIs de modelos é uma competência recorrente em desenvolvimento de aplicações com IA. Também envolve autenticação, limites de requisição, tratamento de erros, latência e custos.

Variáveis de ambiente, .env e Secrets

- Variáveis de ambiente são uma forma de fornecer configurações e credenciais à aplicação sem escrevê-las diretamente no código. O arquivo .env é comum no desenvolvimento local; serviços em nuvem normalmente oferecem mecanismos de secrets.
- Chaves de API não devem ser publicadas no código ou em repositórios. Separar segredo e código reduz riscos e facilita usar configurações diferentes em desenvolvimento e produção.
- O projeto usa **python-dotenv** para carregar o .env e procura **GROQ_API_KEY** nos Secrets do Streamlit ou nas variáveis de ambiente.

```
load_dotenv()
groq_api_key = os.environ.get("GROQ_API_KEY")
```

Ligação com o mercado de trabalho

Gerenciamento seguro de credenciais é uma prática básica de desenvolvimento profissional, DevOps e segurança de aplicações.

Desenvolvimento: construímos o chatbot.
Ambiente de execução: lugar onde o chatbot está rodando.
Deploy: processo de colocar o chatbot nesse ambiente.
DevOps: práticas que ajudam a organizar, automatizar e tornar confiável e seguro todo esse processo.

PyPDFLoader

- Documentos precisam ser convertidos para uma forma que o programa consiga processar. O loader faz a ponte entre o arquivo PDF e as etapas seguintes do RAG.
- PyPDFLoader é um carregador de documentos PDF utilizado no ecossistema LangChain. Ele extrai o conteúdo textual do PDF e o transforma em objetos que podem seguir pelo pipeline.

```
loader = PyPDFLoader(pdf_path)
documentos = loader.load()
```

Ligação com o mercado de trabalho

Carregamento e preparação de documentos são tarefas comuns em pipelines de busca semântica, RAG e automação documental.

Streamlit

- Streamlit é um framework Python voltado à criação rápida de aplicações web interativas, especialmente para dados, Machine Learning e protótipos de IA.
- Permite transformar um script Python em uma interface utilizável sem exigir que o desenvolvedor construa toda a camada de front-end com HTML, CSS e JavaScript.
- Foi usado para criar a página do assistente, o campo de chat, histórico das mensagens, mensagens de erro, spinner e apresentação das respostas.

```
st.chat_input("Digite sua dúvida sobre o edital...")
st.chat_message("assistant")
```

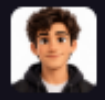
🎓 Assistente Virtual Trabalho Garantido

Faça perguntas sobre o edital do programa e tire suas dúvidas com base nos documentos oficiais.

quem pode participar do programa

🎓 Assistente Virtual Trabalho Garantido

Faça perguntas sobre o edital do programa e tire suas dúvidas com base nos documentos oficiais.



quem pode participar do programa trabalho garantido



Quem pode participar do Programa Trabalho Garantido da FMU

De acordo com o trecho do edital que você enviou, o programa é destinado a:

1. **Alunos de graduação** das instituições participantes (Centro Universitário FMU e FIAM-FAAM).
2. O participante deve estar **regularmente matriculado** no curso de graduação em que está cursando ou em **área correlata** à sua formação.
3. A participação é **estritamente voluntária** – ou seja, o aluno decide se quer aderir ao programa e aceita todas as condições previstas no regulamento.

Digite sua dúvida sobre o edital...

Ligação com o mercado de trabalho

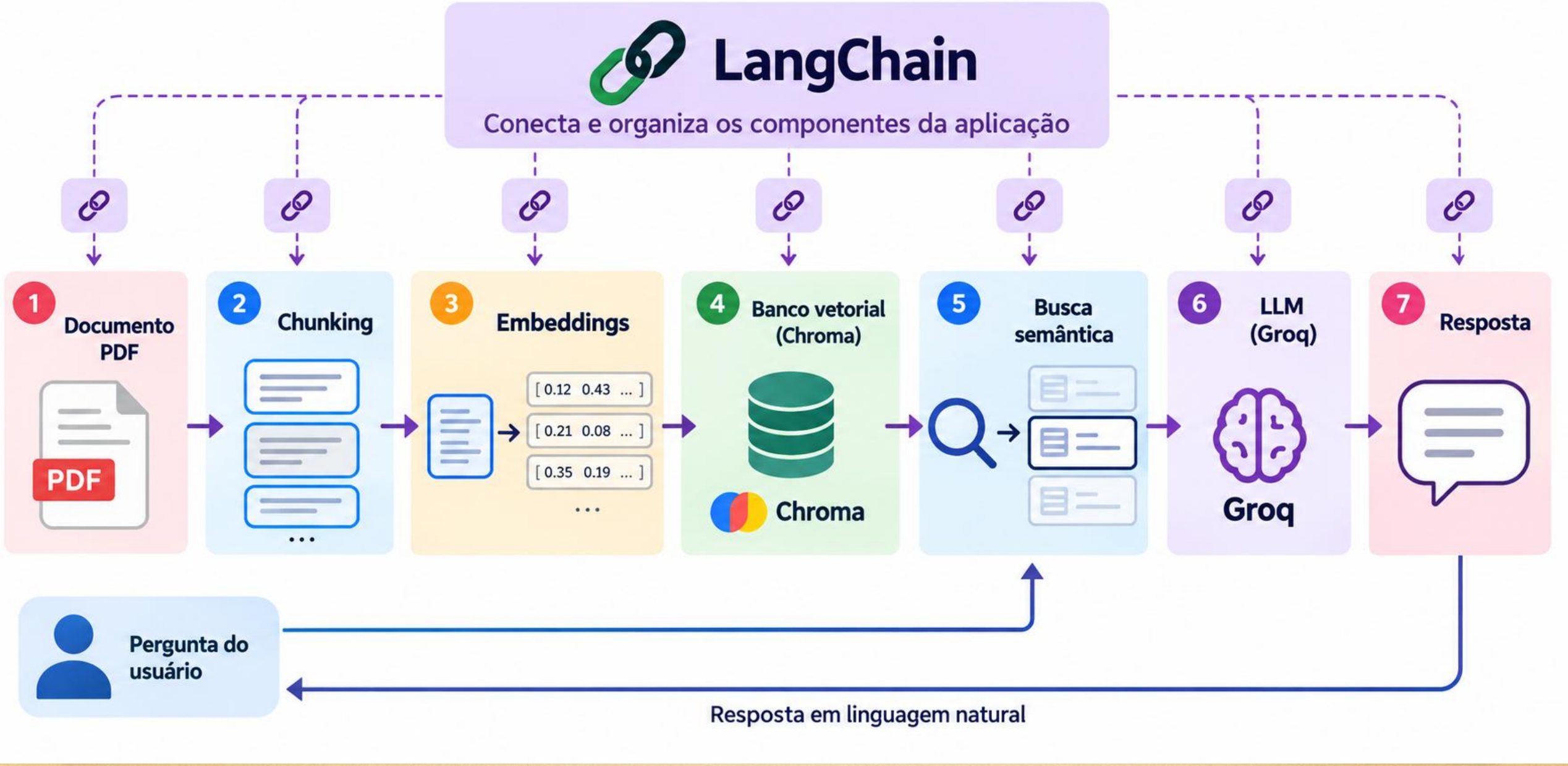
É bastante útil em protótipos, demonstrações, dashboards e aplicações internas de IA. No mercado, é comum em provas de conceito e validação rápida de soluções.

LangChain

- LangChain é um framework para construir aplicações que combinam modelos de linguagem com documentos, recuperadores, bancos vetoriais, prompts e ferramentas.
- Ele fornece abstrações que simplificam a integração entre componentes de uma aplicação de IA generativa.
- No projeto, componentes do ecossistema LangChain são usados para carregar o PDF, dividir textos, trabalhar com Chroma, embeddings e conectar o código ao modelo servido pela Groq.

Ligação com o mercado de trabalho

Conhecer frameworks de orquestração ajuda no desenvolvimento de aplicações de IA. No mercado, porém, é igualmente importante compreender os conceitos por trás do framework para não depender apenas das abstrações.


LangChain
Conecta e organiza os componentes da aplicação
1
Documento PDF
PDF
2
Chunking
3
Embeddings
[0.12 0.43 ...]
[0.21 0.08 ...]
[0.35 0.19 ...]
4
Banco vetorial (Chroma)
Chroma
5
Busca semântica
6
LLM (Groq)
Groq
7
Resposta
Pergunta do usuário
Resposta em linguagem natural

O que é LangChain?

LangChain é um ***framework*** em Python que fornece código pronto (classes e métodos) para integrar e organizar os diferentes componentes de aplicações com modelos de linguagem.

Ele facilita a conexão entre carregamento de documentos, chunking, embeddings, banco vetorial, recuperação de informações e acesso ao LLM.

LangChain

Papel do LangChain

- Fornece classes e métodos já implementados.
- Integra diferentes tecnologias (LLM, embeddings, banco vetorial etc.).
- Padroniza os formatos de entrada e saída.
- Facilita a construção do fluxo da aplicação.
- Permite montar pipelines (chains).
- Não é o LLM, nem o banco vetorial, nem o modelo de embeddings.

Analogia

LangChain é como um **maestro** que coordena os instrumentos da orquestra. Cada tecnologia (PDF, embeddings, Chroma, Groq, LLM etc.) é um instrumento, e o LangChain fornece as partituras (classes e métodos) para que eles toquem em conjunto.



Pipeline do projeto usando LangChain

1. **Carregamento do documento** — PyPDFLoader
2. **Chunking** — RecursiveCharacter TextSplitter
3. **Embeddings** — HuggingFaceEmbeddings all-MiniLM-L6-v2
4. **Banco vetorial** — Chroma
5. **Busca semântica** — similarity_search (recupera chunks relevantes)
6. **Prompt + Contexto** — Trechos recuperados do documento + Pergunta do usuário
7. **LLM** — ChatGroq (gpt-oss-120b)
8. **Resposta** — Em linguagem natural com base no contexto recuperado

Exemplos de código do LangChain no projeto

1. Carregar PDF

```
from langchain_community
 import PyPDFLoader

loader = PyPDFLoader
("edital.pdf")
documentos = loader.load()
```

2. Chunking

```
from langchain_text_splitters
 import RecursiveCharacter
        TextSplitter
splitter = RecursiveCharacter
 TextSplitter(
     chunk_size=1000,
     chunk_overlap=200
)
chunks = splitter.
 split_documents(documentos)
```

3. Embeddings

```
from langchain_huggingface
 import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
  model_name="sentence-
  transformers/all-MiniLM-L6-v2"
```

4. Chroma

```
from langchain_community
 import Chroma

vectorstore = Chroma.
from_documents(
   documents=chunks,
   embedding=embeddings
)
```

5. Busca semântica

```
resultados = vectorstore.
similarity_search(
    pergunta,
    k=4
)
```

6. LLM (Groq)

```
from langchain_groq
 import ChatGroq

llm = ChatGroq(
     model="openai/
     gpt-oss-120b",
     temperature=0
 )
```

O que o LangChain faz por dentro?

- Possui código implementado para ler PDFs, dividir textos, gerar embeddings, armazenar em bancos vetoriais, realizar buscas e se comunicar com LLMs.
- Fornece interfaces padronizadas para integrar diferentes bibliotecas e serviços.
- Facilita a construção de chains (pipelines).

O que o LangChain não é?

- ✗ Não é o modelo de linguagem (LLM).
- ✗ Não é o banco vetorial.
- ✗ Não é o modelo de embeddings.
- ✗ Não conecta tudo automaticamente sem configuração.
- ✗ O programador define o fluxo e utiliza as classes e métodos do LangChain.

Vantagens no projeto

- ✓ Integração facilitada entre diferentes tecnologias.
- ✓ Menos código para tarefas complexas.
- ✓ Flexibilidade para trocar modelos e componentes.
- ✓ Organização do fluxo da aplicação.
- ✓ Padrão amplamente utilizado no mercado.

Cache com st.cache_resource

- Cache é uma técnica para evitar repetir operações caras desnecessariamente. O Streamlit oferece st.cache_resource para reutilizar recursos criados pela aplicação.
- Processar o PDF, gerar embeddings e criar o banco vetorial toda vez que a tela é atualizada seria ineficiente. O cache reduz processamento repetido e melhora a experiência.

```
@st.cache_resource
def inicializar_rag():
    ...
```

Imagine nosso chatbot. Quando a aplicação inicia, ela precisa:

Ler o PDF → dividir em chunks → gerar embeddings → criar/carregar o banco vetorial

Algumas dessas operações podem ser relativamente demoradas. Agora imagine que uma pessoa faça uma pergunta. Em seguinda, outra pessoa entra e faz outra. Se o Streamlit executasse novamente todo esse processamento do edital a cada interação, seria desperdício. Com cache, a lógica pode ser:

- "Eu já carreguei e processei esse edital. Vou reutilizar o resultado."

- **Sem cache**:
  - Pergunta → processa tudo novamente → resposta
- **Com cache**:
  - Pergunta → reutiliza o que já foi processado → resposta

Ligação com o mercado de trabalho

Cache é um conceito transversal em software: aparece em aplicações web, bancos, APIs, Machine Learning e sistemas distribuídos. Saber quando reutilizar resultados pode reduzir latência e custo.

Como as tecnologias se conectam

Etapa	Tecnologia	Função	Resultado
Entrada	Streamlit	Receber a pergunta	Texto do usuário
Fonte	PyPDFLoader	Ler o edital	Documentos
Preparação	Text Splitter	Dividir o conteúdo	Chunks
Representação	Hugging Face / embeddings	Transformar texto em vetores	Vetores semânticos
Armazenamento	Chroma	Guardar e pesquisar vetores	Base vetorial
Recuperação	Retriever	Buscar trechos relevantes	Top 5 chunks
Contexto	Prompt	Unir instruções, fontes e pergunta	Prompt completo
Geração	Groq + GPT-OSS-120B	Gerar resposta	Texto final
Interface	Streamlit	Exibir e manter histórico	Chat interativo

Competências profissionais desenvolvidas

O projeto não ensina apenas uma biblioteca. Ele reúne competências que aparecem em diferentes funções de tecnologia: programação Python, PLN, uso de modelos pré-treinados, embeddings, recuperação semântica, bancos vetoriais, integração com APIs, segurança de credenciais, criação de interfaces e tratamento de erros.

- Construir pipelines de processamento de documentos.
- Integrar modelos e serviços de IA por API.
- Entender a diferença entre recuperação de informação e geração de linguagem.
- Trabalhar com embeddings e busca semântica.
- Criar protótipos web de IA com Streamlit.
- Separar credenciais do código e configurar ambientes.
- Analisar erros, limites de requisição e indisponibilidade de serviços.
- Pensar em arquitetura: cada componente possui uma responsabilidade.

Evoluções possíveis

A versão atual já demonstra um pipeline RAG completo. A partir dela, o projeto pode evoluir sem abandonar os conceitos aprendidos.

- Adicionar mais PDFs e outras fontes documentais.
- Adicionar páginas de um site à base de conhecimento.
- Exibir as fontes e páginas utilizadas na resposta.
- Avaliar diferentes modelos de embeddings.
- Comparar estratégias de chunking e valores de k.
- Persistir o banco vetorial em vez de reconstruí-lo.
- Adicionar consulta a bancos SQL ou APIs externas.
- Criar ferramentas para o LLM e avançar gradualmente para arquiteturas com agentes.
- Adicionar autenticação, logs, avaliação de respostas e controles de segurança para cenários de produção.

Conclusão

O projeto mostra uma mudança importante em relação aos chatbots baseados apenas em classificação de intenções. Aqui, o sistema recupera conhecimento externo e fornece esse conhecimento a um modelo de linguagem capaz de formular respostas. O valor pedagógico está justamente em compreender que um chatbot moderno não é apenas um LLM: ele é uma arquitetura composta por fonte de dados, processamento, embeddings, recuperação, prompt, modelo, API e interface.

Compreender cada componente separadamente permite ao aluno substituir tecnologias no futuro sem perder a visão do todo. Chroma pode ser trocado por outro mecanismo vetorial; Groq por outro provedor; GPT-OSS por outro LLM; Streamlit por outra interface. Os produtos mudam, mas os conceitos arquiteturais permanecem.